FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	*Calvin Josh D. Garcia*
Section:	*AN43*
Professor:	
Date:	

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. *Construct a simple knowledge base using Python data structures.*
2. *Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.*
3. *Build a simple Case-Based Reasoning (CBR) system using similarity matching.*
4. *Compare how RBR and CBR behave when solving the same type of problem.*
5. *Document observations and insights on KR, RBR, and CBR.*

Materials Needed

1. Hardware

- *A laptop or desktop computer*
- *Stable internet connection (for research, documentation, submission)*

2. Software

A programming environment:

- *Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)*
- *Any text editor (Notepad++, Sublime, VS Code) if needed*

3. File Formats & Tools

Word processor for documentation:

- *MS Word, Google Docs, or LibreOffice*
- *Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)*

General Instructions

1. *This* **Technical Assessment (TA3)** *must be completed by Pair.*
2. *Include clear comments in your code to describe its logic.*
3. *Each program should display the correct input, process, and output.*
4. *Ensure that your script displays clear and readable outputs.*
5. *Screenshot of program output for RBR*
6. *Screenshot of program output for CBR*
7. *Reflection answers*
8. *Short description of your knowledge base and use case*
9. *Compile all Python codes and corresponding outputs into one PDF file.*
10. *Save it as* **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. *Submit the* **python file** *and* **PDF** *file.*

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- *5 facts about an object (e.g., weather, appliances, students, medical symptoms)*
- *3 categories (e.g., temperature, status, presence, actions)*

Example structure to follow:

```
facts = {
   "temperature": 32,
   "is_raining": False,
   "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
   if facts["temperature"] > 30:
      return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. *Reads the knowledge base*
2. *Checks each rule*
3. *Returns all actions inferred*

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, *each containing:*

- *A set of problem features*
- *A matching solution*

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. *Which system (RBR or CBR) was easier to implement? Why?*
2. *Which is more flexible in solving unstructured problems?*
3. *Which is better for situations with continuously changing data?*
4. *Examples of real-world applications for both.*

Your Code - Laboratory Tasks ⇩

```
# PART 1 — Knowledge Representation (KR) Setup
# Task 1A: Knowledge Base storing facts and categories regarding smart
home/weather
knowledge_base = {
    "temperature": 32,      # Category: Temperature (in Celsius)
    "is_raining": True,     # Category: Presence/Weather Status (Boolean)
    "humidity": 85,         # Category: Status/Atmospheric (Percentage)
    "power_status": True,   # Category: Status (Boolean)
    "light_level": "dark"   # Category: Status/Environment ("dark" or "bright")
```

```
}


# PART 2 — Implement Rule-Based Reasoning (RBR)
# Task 2A: Define IF-THEN Rules as functions

def rule_hot(facts):
    """Rule 1: If temperature is above 30°C, suggest turning on the aircon."""
    if facts.get("temperature", 0) > 30:
        return "Turn on the aircon"
    return None

def rule_rain(facts):
    """Rule 2: If it is raining, suggest bringing an umbrella."""
    if facts.get("is_raining", False):
        return "Bring umbrella"
    return None

def rule_lighting(facts):
    """Rule 3: If environment is dark and power is on, suggest turning on
lights."""
    if facts.get("light_level") == "dark" and facts.get("power_status", False):
        return "Turn on indoor lights"
    return None

# List of all active rules for the inference engine
rules = [rule_hot, rule_rain, rule_lighting]

# Task 2B: Inference Engine
def rbr_inference_engine(facts, rule_set):
    """
    Inference Engine that evaluates all rules against current facts
    and returns a list of inferred actions.
    """
    inferred_actions = []
    for rule in rule_set:
        action = rule(facts)
        if action:
            inferred_actions.append(action)
    return inferred_actions
```

```
# PART 3 — Implement Case-Based Reasoning (CBR)
# Task 3A: Build a Case Base with past problems and solutions
case_base = [
    {
        "id": 1,
        "problem": {"won_t_turn_on": True, "noise": False, "overheating":
False},
        "solution": "Check power cable and outlet connection."
    },
    {
        "id": 2,
        "problem": {"won_t_turn_on": True, "noise": True, "overheating": True},
        "solution": "Clean internal cooling fan and replace thermal paste."
    },
    {
        "id": 3,
        "problem": {"won_t_turn_on": False, "noise": True, "overheating":
True},
        "solution": "Inspect fan blades for physical obstructions."
    }
]

def calculate_similarity(problem1, problem2):
    """Helper method to calculate similarity score based on matching feature
values."""
    matching_features = 0
    total_features = len(problem1)

    for key in problem1:
        if key in problem2 and problem1[key] == problem2[key]:
            matching_features += 1

    return matching_features / total_features if total_features > 0 else 0

# Task 3B & 3C: Retrieve and Reuse Steps
def cbr_retrieve_and_reuse(new_problem, cases):
    """Finds the most similar past case and returns its solution."""
    best_match = None
    highest_similarity = -1.0
```

```
    for case in cases:
        sim_score = calculate_similarity(new_problem, case["problem"])
        if sim_score > highest_similarity:
            highest_similarity = sim_score
            best_match = case

    return best_match, highest_similarity

# Task 3D: Revise Step
def cbr_revise(solution):
    """Allows user/student to manually adjust the solution or keep it
unchanged."""
    print(f"\n[CBR - Revise Step] Suggested Solution: '{solution}'")
    user_input = input("Would you like to revise this solution? (yes/no):
").strip().lower()

    if user_input == 'yes':
        new_solution = input("Enter the revised solution: ").strip()
        return new_solution
    return solution

# Task 3E: Retain Step
def cbr_retain(cases, new_problem, final_solution):
    """Stores the newly solved case into the case base for future reference."""
    new_case_id = len(cases) + 1
    new_case = {
        "id": new_case_id,
        "problem": new_problem,
        "solution": final_solution
    }
    cases.append(new_case)
    print(f"[CBR - Retain Step] New Case #{new_case_id} retained in Case Base
successfully.")


# MAIN EXECUTION / OUTPUT DEMONSTRATION
if __name__ == "__main__":
    # --- RBR DEMO ---
    actions = rbr_inference_engine(knowledge_base, rules)
    print("Rule-Based Reasoning Actions:")
    for action in actions:
```

```
        print(f" - {action}")

    # --- CBR DEMO ---
    # Define a new problem query
    new_problem_query = {"won_t_turn_on": True, "noise": True, "overheating":
False}
    print(f"New Problem Input: {new_problem_query}")

    # 1. Retrieve & Reuse
    matched_case, similarity = cbr_retrieve_and_reuse(new_problem_query,
case_base)
    print(f"Most Similar Case Found: Case #{matched_case['id']} (Similarity
Score: {similarity * 100:.1f}%)")
    reused_solution = matched_case["solution"]

    # 2. Revise
    revised_solution = cbr_revise(reused_solution)
    print(f"Final Solution to Apply: {revised_solution}")

    # 3. Retain
    cbr_retain(case_base, new_problem_query, revised_solution)
```